Universidad Mayor de San Andrés

Facultad de Ciencias Puras y Naturales

Carrera de Informática

LABORATORIO NRO # 2

Apellidos y Nombres: Mamani Challapa Romulo Mateo

CI: 9951413

Carrera: Informática

Materia: Programación 2

Paralelo: “D”

Docente: Lic. Jhonny Roberto Felipez Andrade

Fecha de Entrega: 16 de marzo del 2026

1. *La clase MiPunto.* Diseñe una clase llamada MiPunto para representar un punto con coordenadas x e y. La clase contiene:

   a) Los atributos x e y que representan las coordenadas con métodos getter.

   b) Un constructor sin argumentos que crea un punto (0, 0).

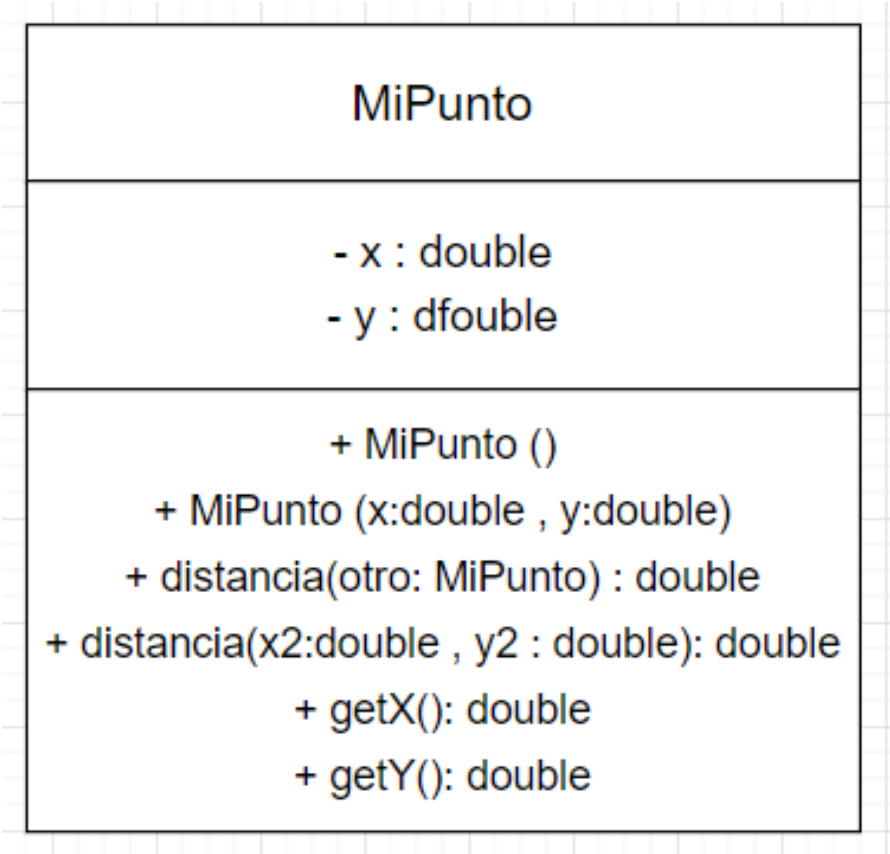
   c) Un constructor que construye un punto con las coordenadas especificadas.

   d) Un método llamado **distancia** que retorna la distancia desde este punto hasta un punto especificado del tipo MiPunto.

   e) Un método llamado **distancia** que retorna la distancia desde este punto hasta otro punto con las coordenadas x e y especificadas.

   Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba que cree los puntos (0, 0) y (10, 30.5) y muestre la distancia entre ellos.

//DIAGRAMA UML

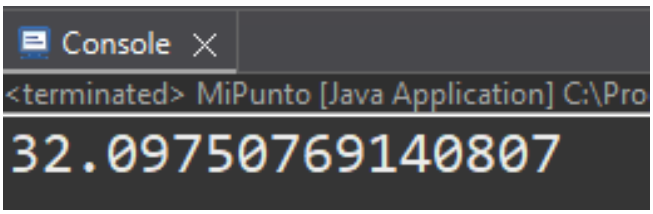


//CODIGO JAVA

```
class MiPunto {

     //a) Los atributos x e y que representan las coordenadas con metodos getter.
     private double x;
     private double y;

     //b) Un constructor sin argumentos que crea un punto (0, 0).

     public MiPunto(){

     }

     //c) Un constructor que construye un punto con las coordenadas
especificadas.

     public MiPunto(double x,double y) {
          this.x = x;
          this.y = y;
     }





     /*d) Un metodo llamado distancia que retorna la
          distancia desde este punto hasta un
          punto especificado del tipo MiPunto.
    */




     public double distancia(MiPunto otro) {

          double dist = Math.sqrt( (Math.pow((otro.x - this.x),2)) + (Math.pow((
otro.y - this.y ),2)) ) ;
          return dist;
     }
```

```
     /*
     e) Un metodo llamado distancia que retorna la
     distancia desde este punto hasta otro
     punto con las coordenadas x e y especificadas.
    */

     public double distancia(double x2 , double y2) {

          double dist = Math.sqrt( (Math.pow((x2 - this.x),2)) + (Math.pow(( y2 -
this.y ),2)) ) ;
          return dist;
     }


     //getters x e y
     public double getX() {
          return x;
     }

     public double getY() {
          return y;
     }



 //Main

     public static void main(String[] args) {
          /*Escriba un programa de prueba que
          cree los puntos (0, 0) y (10, 30.5) y
          muestre la distancia entre ellos.
          */

          MiPunto m1 = new MiPunto();
          MiPunto m2 = new MiPunto(10,30.5);

          System.out.println(m1.distancia(m2));

     }

}
```

//EJECUCION



2. *Algebra Vectorial.* Se quiere construir un programa orientado a objetos con sobrecarga de funciones. Para la solución del problema se debe dibujar el diagrama de clases, acompañado del respectivo código de solución.

   El programa debe: (1) determinar si dos vectores son perpendiculares, (2) determinar si dos vectores son paralelos, (3) determinar la proyección de dos vectores, (4) determinar el componente de dos vectores. Para ello genere la clase AlgebraVectorial con la sobrecarga de constructores y sobrecargando los siguientes métodos:

   a) Perpendicular. Si el vector **a** es ortogonal o perpendicular a **b**: $|\mathbf{a}+\mathbf{b}| = |\mathbf{a}-\mathbf{b}|$

   b) Perpendicular. Si el vector **a** es mutuamente ortogonal a **b**: $|\mathbf{a}-\mathbf{b}| = |\mathbf{b}-\mathbf{a}|$

   c) Perpendicular. Si el vector **a** es ortogonal a **b**: $\mathbf{a}\cdot\mathbf{b} = 0$

   d) Perpendicular. Si el vector **a** es ortogonal a **b**: $|\mathbf{a}+\mathbf{b}|^2 = |\mathbf{a}|^2 + |\mathbf{b}|^2$

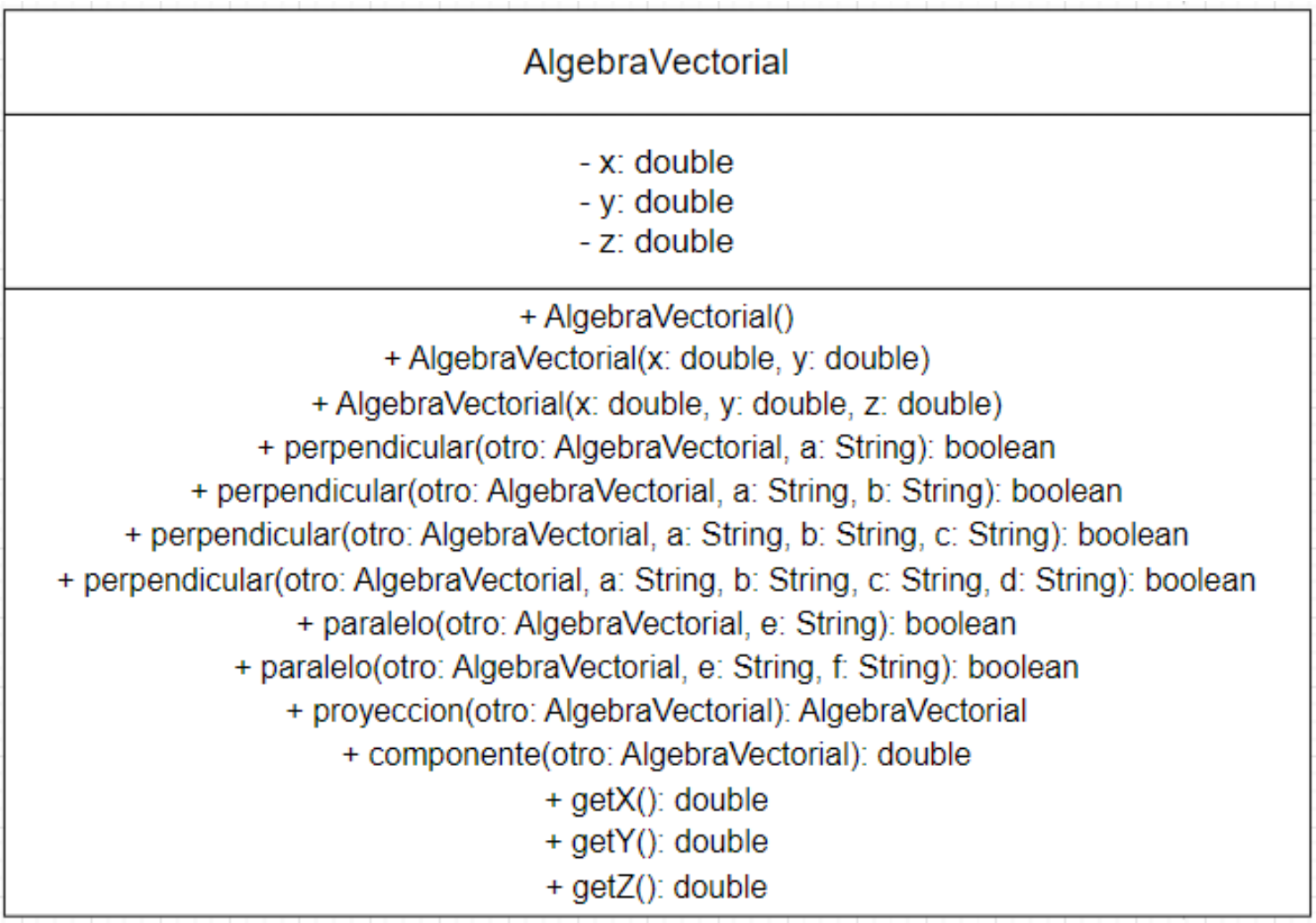
   e) Paralela. Si el vector **a** es paralela a **b**: $\mathbf{a} = r\mathbf{b}$

   f) Paralela. Si el vector **a** es paralela a **b**: $\mathbf{a}\times\mathbf{b} = 0$

   g) Proyeccion_de_a_sobre_b. La proyección ortogonal de **a** sobre **b**: $Proy_b\,\mathbf{a} = \dfrac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{b}|^2}\mathbf{b}$

   h) Componente_de_a_en_b. El componente de **a** en la dirección de **b**: $Comp_b\,\mathbf{a} = \dfrac{\mathbf{a}\cdot\mathbf{b}}{|\mathbf{b}|}$

//DIAGRAMA UML



//CODIGO JAVA

```
package algebraLienal;
class AlgebraVectorial {

	private double x;
	private double y;
	private double z;

	//Constructores sobrecargados

	public AlgebraVectorial(){

	}

	public AlgebraVectorial(double x,double y){
		this.x = x;
		this.y = y;
		this.z = 0;
	}

	public AlgebraVectorial(double x,double y,double z){
		this.x = x;
		this.y = y;
		this.z = z;
	}

	// INCISOS DE PERPENDICULARIDAD
	// inciso a) |a + b| = |a - b|

	public boolean perpendicular(AlgebraVectorial otro, String a) {
	    double suma = Math.sqrt(Math.pow(x + otro.x, 2) + Math.pow(y + otro.y,
2) + Math.pow(z + otro.z, 2));
	    double resta = Math.sqrt(Math.pow(x - otro.x, 2) + Math.pow(y - otro.y,
2) + Math.pow(z - otro.z, 2));

	    return suma == resta;
	}


	// inciso b) |a - b| = |b - a|

	public boolean perpendicular(AlgebraVectorial otro, String a, String b) {
	    double restaAB = Math.sqrt(Math.pow(x - otro.x, 2) + Math.pow(y -
otro.y, 2) + Math.pow(z - otro.z, 2));
	    double restaBA = Math.sqrt(Math.pow(otro.x - x, 2) + Math.pow(otro.y -
y, 2) + Math.pow(otro.z - z, 2));

	    return restaAB == restaBA;
	}
```

```
//inciso c)  a · b = 0
public boolean perpendicular(AlgebraVectorial otro, String a,String b,String
c) {

     if(((x*otro.x)+(y*otro.y)+(z*otro.z)) == 0) {
          return true;
     }
     return false;
}

// inciso d) |a + b|^2 = |a|^2 + |b|^2

public boolean perpendicular(AlgebraVectorial otro, String a, String b,
String c, String d) {
     double sumaCuadrada = Math.pow(x + otro.x, 2) + Math.pow(y + otro.y, 2)
+ Math.pow(z + otro.z, 2);
     double pitagoras = (x*x + y*y + z*z) + (otro.x*otro.x + otro.y*otro.y +
otro.z*otro.z);
     return sumaCuadrada == pitagoras;
}



//INCISOS DE PARALELISMO

// inciso e) a = rb
public boolean paralelo(AlgebraVectorial otro, String e) {
    // Si la división de sus componentes es igual, son paralelos
    double rx = (double)x / otro.x;
    double ry = (double)y / otro.y;
    double rz = (double)z / otro.z;
    return (rx == ry && ry == rz);
}


// inciso f) a × b = 0

public boolean paralelo(AlgebraVectorial otro, String e, String f) {
    double cx = (y * otro.z) - (z * otro.y);
    double cy = (z * otro.x) - (x * otro.z);
    double cz = (x * otro.y) - (y * otro.x);
    // Si todos los componentes del producto cruz son 0
    return (cx == 0 && cy == 0 && cz == 0);
}
```

```
    // PROYECCIÓN Y COMPONENTE

    // inciso g) Proyeccion de a sobre b

    public AlgebraVectorial proyeccion(AlgebraVectorial otro) {
        double productoPunto = (x * otro.x) + (y * otro.y) + (z * otro.z);

        double magnitudBSquared = Math.pow(otro.x, 2) + Math.pow(otro.y, 2) +
Math.pow(otro.z, 2);


        // Protección: No se puede dividir entre cero
        if (magnitudBSquared == 0) {
            System.out.println("Error: No se puede proyectar sobre un vector
nulo.");
            return new AlgebraVectorial(0, 0, 0);
        }

        double escalar = productoPunto / magnitudBSquared;
        return new AlgebraVectorial(escalar * otro.x, escalar * otro.y, escalar
* otro.z);
    }


    // inciso h) Componente de a en b

    public double componente(AlgebraVectorial otro) {
        double productoPunto = (x * otro.x) + (y * otro.y) + (z * otro.z);
        double magnitudB = Math.sqrt(Math.pow(otro.x, 2) + Math.pow(otro.y, 2) +
Math.pow(otro.z, 2));

        if (magnitudB == 0) return 0; // Evitamos el error de división

        return productoPunto / magnitudB;
    }


   // --- MÉTODOS GET ---
   public double getX() { return x; }
   public double getY() { return y; }
   public double getZ() { return z; }
```

// Main

```
     public static void main(String[] args) {
         // Definimos dos vectores para la prueba

          AlgebraVectorial v1 = new AlgebraVectorial(1,2,3);
          AlgebraVectorial v2 = new AlgebraVectorial(2,-1,0);

         System.out.println("--- PRUEBAS DE VECTORES ---");

       //(1) determinar si dos vectores son perpendiculares , usamos el inciso c)

          if(v1.perpendicular(v2,"a","b","c")) {
               System.out.println("Resultado (1): es perpendicular");
          }else {
               System.out.println("Resultado (1): no es perpendicular");
          }

       // (2) Determinar si son paralelos (usando el inciso f)
         if (v1.paralelo(v2, "e", "f")) {
             System.out.println("Resultado (2): Los vectores son PARALELOS.");
         } else {
             System.out.println("Resultado (2): Los vectores NO son paralelos.");
         }

       // (3) Determinar la proyeccion (inciso g)

         AlgebraVectorial proy = v1.proyeccion(v2);
         System.out.println("Resultado (3): La Proyeccion de A sobre B es: (" +
proy.getX() + ", " + proy.getY() + ", " + proy.getZ() + ")");

       // (4) Determinar el componente (inciso h)
         double comp = v1.componente(v2);
         System.out.println("Resultado (4): El Componente de A en la direccion de
B es: " + comp);

     }
}
```

//EJECUCION

```
Console
<terminated> AlgebraVectorial [Java Application] C:\Program Files\Java\jre1.8.0_471\bin\javaw.exe (4 mar 2026, 18:43:22 – 18:43:23 elapsed: 0:00:00.143) [pid: 13
--- PRUEBAS DE VECTORES ---
Resultado (1): es perpendicular
Resultado (2): Los vectores NO son paralelos.
Resultado (3): La Proyeccion de A sobre B es: (0.0, -0.0, 0.0)
Resultado (4): El Componente de A en la direccion de B es: 0.0
```

3. En álgebra vectorial un vector tridimensional se define como: $\mathbf{a} = (a_1, a_2, a_3)$. Además se tiene lo siguiente:

   a) Suma de dos vectores **a** y **b**: $\mathbf{c} = \mathbf{a} + \mathbf{b} = (a_1 + b_1, a_2 + b_2, a_3 + b_3)$

   b) Multiplicación de un escalar r por un vector **a**: $\mathbf{b} = r\mathbf{a} = (r * a_1, r * a_2, r * a_3)$

   c) Longitud de un vector **a**: $|\mathbf{a}| = \sqrt{a_1^2 + a_2^2 + a_3^2}$

   d) Normal de un vector **a**: $\mathbf{b} = \dfrac{\mathbf{a}}{|\mathbf{a}|} = \left(\dfrac{a_1}{|\mathbf{a}|}, \dfrac{a_2}{|\mathbf{a}|}, \dfrac{a_3}{|\mathbf{a}|}\right)$

   e) Producto escalar de **a** y **b**: $\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + a_3b_3$

   f) Producto vectorial de **a** y **b**: $\mathbf{a} \times \mathbf{b} = (a_2b_3 - a_3b_2, a_3b_1 - a_1b_3, a_1b_2 - a_2b_1)$

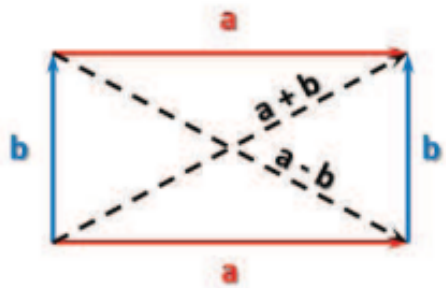


Figura 1: **a** es perpendicular u ortogonal a **b** si las diagonales del paralelogramo formado por **a** y **b** son de igual longitud, es decir, si el paralelogramo es un rectángulo.

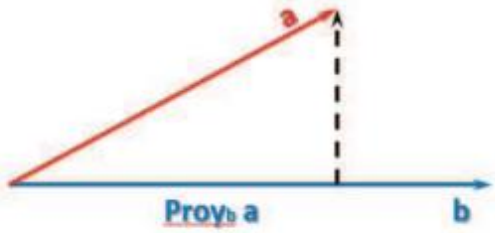
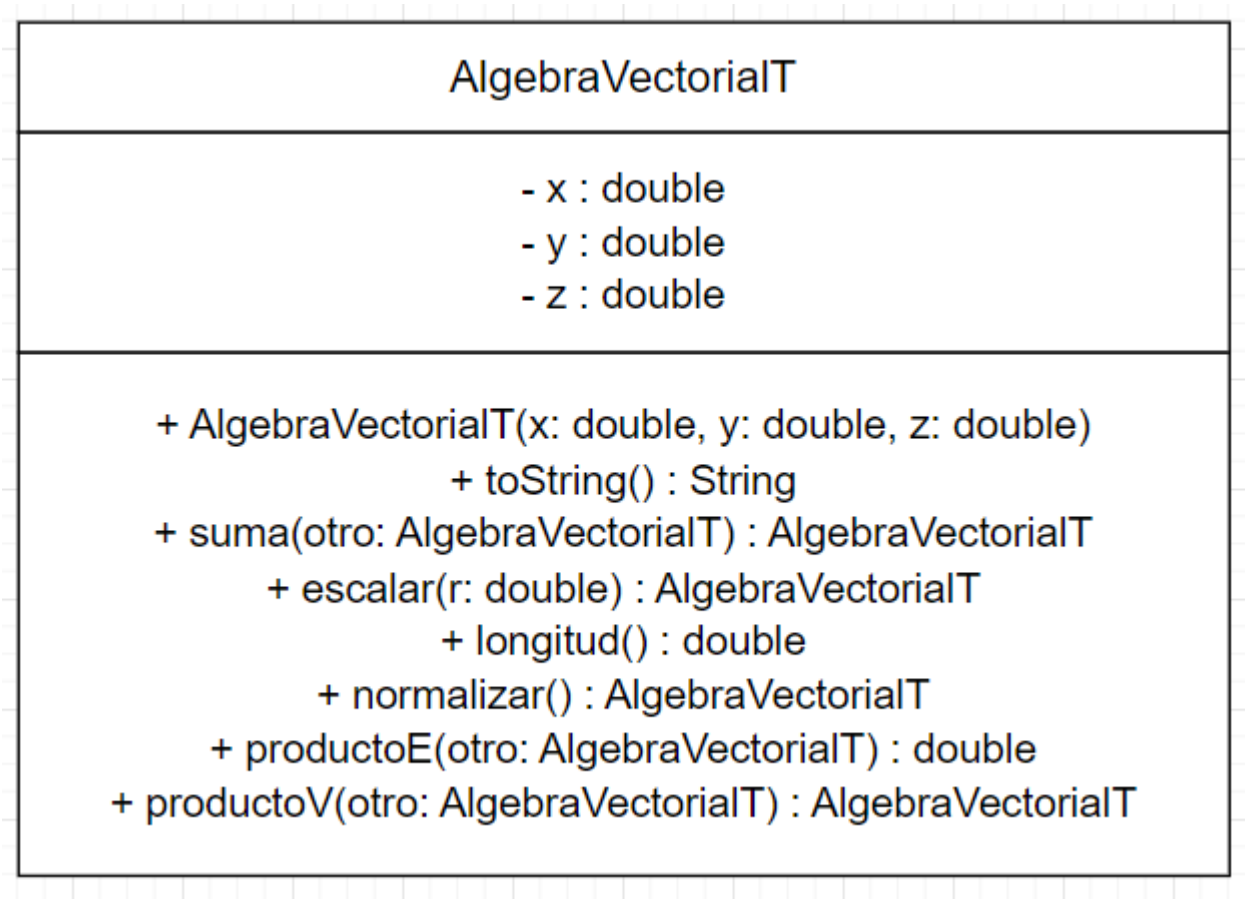


Figura 2: Proyección ortogonal y componentes.

Se quiere construir un programa orientado a objetos con sobrecarga de operadores. Para la solución del problema se debe dibujar el diagrama de clases, acompañado del respectivo código de solución.

//DIAGRAMA UML



//CODIGO JAVA

```
package vectorT;

public class AlgebraVectorialT {

	private double x;
	private double y;
	private double z;

	public AlgebraVectorialT(double x,double y,double z){
		this.x = x;
		this.y = y;
		this.z = z;
	}

	public String toString() {
		return String.format("(%.1f ;"+" %.1f ;"+" %.1f)", x , y , z);
	}
```

```
//sobrecarga de operadores

//a) Suma de vectores

public AlgebraVectorialT suma(AlgebraVectorialT otro) {
     double t = x + otro.x;
     double s = y + otro.y;
     double w = z + otro.z;

     return new AlgebraVectorialT(t,s,w);
}

//b) b) Multiplicacion de un escalar r por un vector a

public AlgebraVectorialT escalar(double r) {
     double t = r * x;
     double s = r * y;
     double w = r * z;

     return new AlgebraVectorialT(t,s,w);
}

//c) longitud de un vector a

public double longitud() {
     return Math.sqrt((Math.pow(x,2))+(Math.pow(y,2))+(Math.pow(z,2)));
}

//d)Normalizar un vector a

public AlgebraVectorialT normalizar() {
     double Long = longitud();
     if(Long != 0) {
          double t = x/Long;
          double s = y/Long;
          double w = z/Long;
          return new AlgebraVectorialT(t,s,w);
     }
     return new AlgebraVectorialT(0,0,0);
}

//e) Producto escalar de a y b

public double productoE (AlgebraVectorialT otro) {
     return (x*otro.x)+(y*otro.y)+(z*otro.z);
}
```

```
     //f)Producto vectorial de a y b
     public AlgebraVectorialT productoV (AlgebraVectorialT otro) {
          double t = (y*otro.z)-(z*otro.y);
          double s = (z*otro.x)-(x*otro.z);
          double w = (x*otro.y)-(y*otro.x);

          return new AlgebraVectorialT(t,s,w);
     }


}
```

//MAIN

```
package vectorT;

class Main {

     public static void main(String[] args) {
          AlgebraVectorialT a = new AlgebraVectorialT(1,2,3);
          AlgebraVectorialT b = new AlgebraVectorialT(1,1,1);
          System.out.println("a = "+ a +"  b = "+b);
          System.out.println("a) a + b = "+a.suma(b));
          System.out.println("b) r=2 , r*a = "+a.escalar(2));
          System.out.println(String.format("c) longitud de a = %.2f",
a.longitud()));


          if(a.longitud() == 0){
              System.out.println("No se puede normalizar el vector
cero.");
          }else{
               System.out.println("d) a/|a| = " + a.normalizar());
          }

          System.out.println(String.format("e) a·b = %.2f",
a.productoE(b)));
          System.out.println("f) axb = "+ a.productoV(b));




     }

}
```

//EJECUCION

```
Console
<terminated> Main (72) [Java Application] C:\Program Files\Java\jre1.8.0_471\bin\javaw.exe  (5 mar 2026, 15:03:3
a = (1,0 ; 2,0 ; 3,0)  b = (1,0 ; 1,0 ; 1,0)
a) a + b = (2,0 ; 3,0 ; 4,0)
b) r=2 , r*a = (2,0 ; 4,0 ; 6,0)
c) longitud de a = 3,74
d) a/|a| = (0,3 ; 0,5 ; 0,8)
e) a·b = 6,00
f) axb = (-1,0 ; 2,0 ; -1,0)
```